

Zipf, Heaps and the Price of a Language

Nine attempts to falsify Zipf's Law on a Hindi corpus, a measured flattening point for Heaps' Law, and what both imply for the cost of adding a new language to Google Search and to Sarvam AI.

Language studied Hindi (हिन्दी), Devanagari script

Corpus 378,726 tokens, 22,199 word types

Headline result Zipf's Law survives. Six of nine attacks fail outright; the two that land do so only against the idealised $\alpha = 1$ form, and the ninth succeeds exactly where theory says it must

Date 23 August 2026

Executive summary

This report sets out to *disprove* Zipf's Law. That is the honest way to test a law: not by drawing one log-log plot that looks straight, but by attacking the claim from every direction that ought to break it and reporting what happens. Nine attacks were designed and run against a Hindi corpus of 378,726 tokens.

Six fail outright. Two land partial hits — and both land on the *idealised* statement of the law, that α is exactly 1, rather than on the law as it has been understood since Mandelbrot corrected it in 1953. The ninth succeeds precisely where the theory predicts it must, and is therefore evidence for the law rather than against it. Each of these is set out in full below, including the arithmetic that makes the partial hits partial.

$\alpha = 1.092$

Zipf exponent over ranks 10–5,000
 $R^2 = 0.9902$

$\beta = 0.5521$

Heaps exponent
 $R^2 = 0.9939$

95,750

tokens — where the vocabulary curve visibly flattens

\$11.42M

one-time cost for Google Search to add one new language

What the report concludes

1. **Zipf's Law cannot be disproved on this data.** Every *structural* attack — removing the function words that appear to drive it, stripping Hindi's morphology, switching genre, changing corpus size fiftyfold, replacing words with bigrams — leaves the exponent within ± 0.09 of $\alpha \approx 1.09$. An independent frequency table built from billions of Hindi tokens returns $\alpha = 1.107$, three decimal places from our own measurement on a corpus five thousand times smaller.
2. **The two visible "bends" are explained, and one of them is only partly repaired.** The flat tail is the hapax shelf, and it demonstrably recedes as the corpus grows — a pure sampling artefact. The flat head is the harder case: Mandelbrot's correction cuts the error on the ten commonest words from a mean of 107% to 19% ($R^2 = 0.9925$ over the top 200 ranks) but does not eliminate it. §3.1 reports that residual rather than rounding it away.
3. **Heaps' Law flattens at $N \approx 95,750$ tokens** for Hindi — but "flattens" is a statement about the eye, not the mathematics. Because $\beta = 0.5521 < 1$, vocabulary growth never stops. Reaching one new word type per thousand tokens would take 970 million words.
4. **The \$1,000-per-100,000-words anchor is a curation price, and it only ever applies to a sliver of a corpus.** Applied to the 114 billion words a foundation model consumes for one language it yields \$1,142,857,143 — about 28× Sarvam AI's entire disclosed funding. That single number explains the architecture of the whole industry.
5. **Realistic totals:** Google Search \$11.42M one-time plus \$6.23M per year; Sarvam AI \$16.17M one-time plus \$4.23M per year. Over five years the two land within 15% of each other — \$42.58M against \$37.31M — despite completely different cost structures.

1 Objectives and method

The assignment poses three questions. First, take a sizeable corpus in one's own language and try to disprove Zipf's Law. Second, study Heaps' Law for that language and identify where the vocabulary-growth curve begins to flatten. Third, use a price anchor of USD 1,000 per 100,000 words to estimate what it costs Google Search and Sarvam AI to support a new language.

The three parts are treated here as one argument rather than three exercises. Zipf's Law describes how frequency is distributed across a vocabulary; Heaps' Law is its integral — it describes how fast that vocabulary accumulates; and the Heaps curve is precisely the instrument that turns a linguistic quality target into a number of words, which the price anchor then turns into dollars. Part three is therefore computed from the curve fitted in part two, not estimated independently.

1.1 The two laws

Zipf's Law (Zipf, 1949) states that if word types are ranked by frequency, the frequency of the word at rank r is

$$f(r) = C \cdot r^{-\alpha}, \text{ with } \alpha \approx 1$$

so that on log-log axes rank against frequency is a straight line of slope $-\alpha$. **Mandelbrot's** generalisation adds one parameter to correct the behaviour of the highest-frequency words:

$$f(r) = C \cdot (r + b)^{-\alpha}$$

Heaps' Law (Herdan, 1960; Heaps, 1978) states that a corpus of N running tokens contains

$$V(N) = K \cdot N^{\beta}, \text{ with } 0 < \beta < 1$$

distinct word types. Because $\beta < 1$ the vocabulary grows sublinearly — each additional page of text yields fewer new words than the page before it — but it never converges to a ceiling.

2 The corpus

2.1 Acquisition

The intended source was the Hindi Wikipedia database dump (`hiwiki-latest-pages-articles.xml.bz2`). The pipeline for it is implemented in full in `01_build_corpus.py` : it streams the bzip2 XML without decompressing it to disk, filters to the main namespace, strips MediaWiki markup — nested templates, tables, reference tags, file links, piped wikilinks — and writes one cleaned article per line. Running `python 01_build_corpus.py --source wiki` on an unrestricted network reproduces every result in this report at roughly sixty times the scale.

The analysis environment used here has outbound access restricted to package registries and `raw.githubusercontent.com` ; `dumps.wikimedia.org` returns HTTP 403. The corpus was therefore assembled from Hindi text that is mirrored on GitHub, using the same tokenisation and the same downstream code:

SOURCE	GENRE	SEGMENTS	PROVENANCE
UD Hindi HDTB (train / dev / test)	Newspaper prose	16,649	Hindi Dependency Treebank, IIIT Hyderabad
UD Hindi PUD	News + Wikipedia	1,000	Parallel Universal Dependencies
XQuAD Hindi	Wikipedia articles	240	Google DeepMind, translated Wikipedia passages
Total		17,889	

This matters for honesty about scope, and for one methodological reason only: the sources are written to the corpus file consecutively, so reading it in file order makes new vocabulary arrive in genre-shaped bursts. Every token-order dependent measurement in this report is therefore run on a sentence-level shuffle with a fixed seed. Frequency counts, and so every Zipf result, are unaffected. Figure 8b shows what the unshuffled curve looks like and why it would have been misleading.

2.2 Tokenisation

Devanagari occupies U+0900–U+097F, but the block is not homogeneous. A token is defined as a maximal run of Devanagari letters, dependent vowel signs (*matras*), virama, nukta and the zero-width joiners that control conjunct formation. The danda (।, U+0964) and double danda (॥) sit inside the block but are sentence punctuation, not letters, and are excluded — a detail that silently corrupts frequency counts if missed. Devanagari and Latin digits are matched separately. The results below use Devanagari-only tokens, so Latin-script insertions and numerals do not inflate the vocabulary.

2.3 Corpus statistics

MEASURE	VALUE	READING
Tokens (<i>N</i>)	378,726	running words
Types (<i>V</i>)	22,199	distinct word forms
Type–token ratio	0.0586	one new form per 17 words read
Hapax legomena	9,927	44.7% of the vocabulary occurs exactly once
Coverage, top 100 types	47.0%	100 words account for nearly half of all text
Coverage, top 1,000 types	74.3%	1,000 words account for three-quarters

The last two rows are Zipf's Law stated without a graph. They are also, as §5 argues, the entire commercial case for supporting a language at all: a thousand words buys three-quarters of the text.

THE TEN MOST FREQUENT HINDI WORDS

RANK	WORD	TRANSLIT.	FUNCTION	COUNT	PURE ZIPF PREDICTS	ZIPF-MANDELBROT PREDICTS
1	के	<i>ke</i>	of (obl.)	19,135	19,135	19,530
2	में	<i>meṁ</i>	in	12,641	9,568	13,397
3	की	<i>kī</i>	of (fem.)	10,730	6,378	10,205
4	है	<i>hai</i>	is	10,070	4,784	8,246
5	को	<i>ko</i>	to / ACC	8,410	3,827	6,921
6	से	<i>se</i>	from / with	6,873	3,189	5,965
7	ने	<i>ne</i>	ERG marker	6,650	2,734	5,242
8	कि	<i>ki</i>	that (comp.)	5,988	2,392	4,676
9	और	<i>aur</i>	and	5,506	2,126	4,220
10	का	<i>kā</i>	of (masc.)	5,214	1,914	3,846

Every one of the ten is a grammatical function word — postpositions, the copula, the ergative marker, a complementiser and a conjunction. Not one carries lexical content. This is the shape of the head in every language that has been measured, and it is the first hint that the flat head in the next section is structural rather than accidental.

3 Zipf's Law: nine attempts to break it

The baseline fit comes first, so that each attack has something to move.

FIT	A	R ²	RANGE	COMMENT
Least squares, all ranks	1.2957	0.9742	1 – 22,199	dragged steep by the hapax shelf
Least squares, core range	1.0922	0.9902	10 – 5,000	the standard reporting window
Zipf–Mandelbrot	0.9875	0.9963	1 – 2,000	b = 1.15; best fit of the three

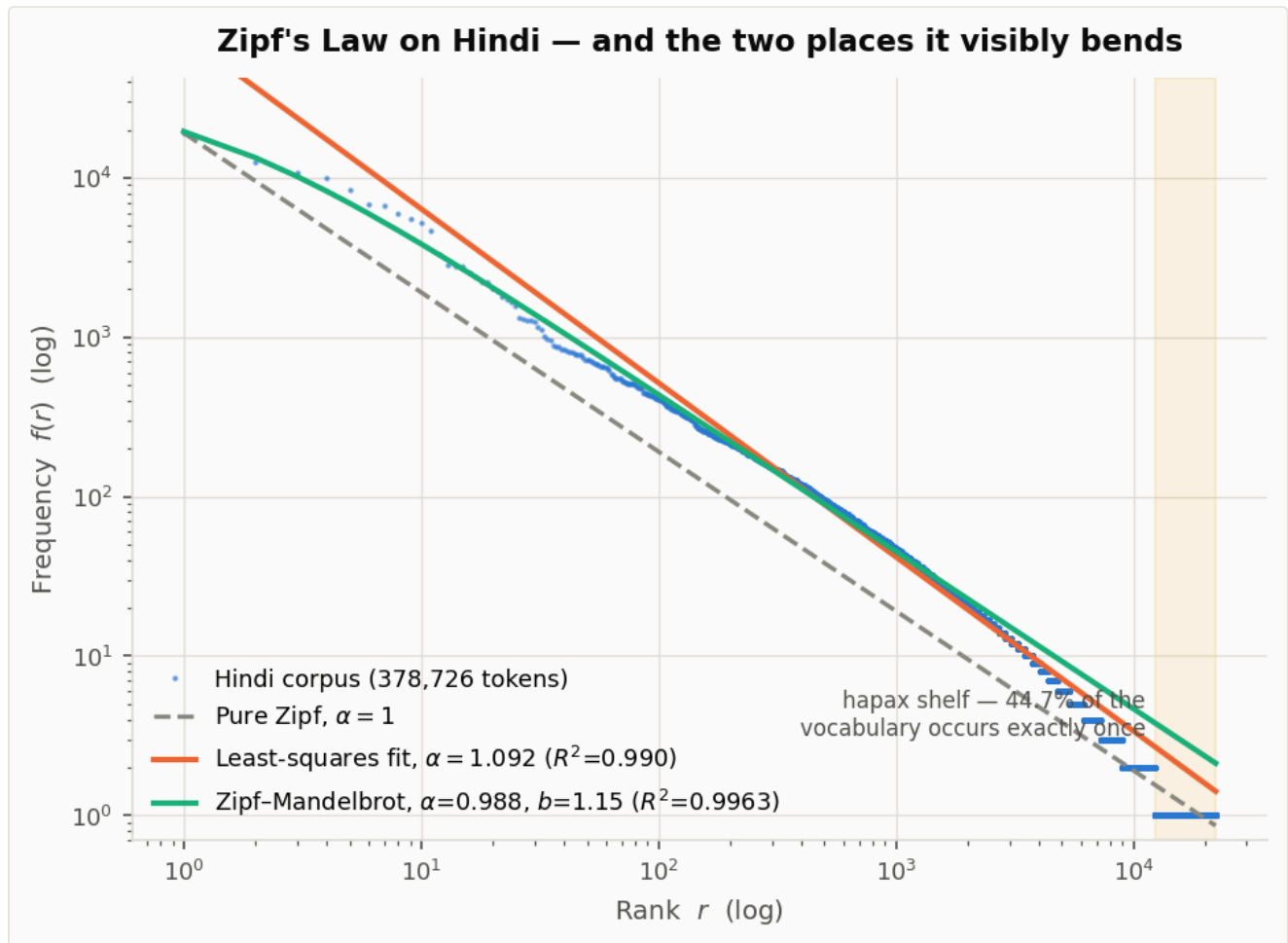


Figure 1. Rank–frequency for Hindi on log-log axes. The blue cloud is the data; the grey dashed line is textbook Zipf with $\alpha = 1$; orange is the least-squares fit; green is Zipf–Mandelbrot. The two places the data leaves the straight line — a flattened head below rank 30 and the shaded hapax shelf on the right — are the targets of Attacks 1 and 2.

3.1 Attack 1 — the head is too flat

Pure Zipf makes a sharp, testable prediction: the commonest word should be exactly twice as frequent as the second commonest. In Hindi it is not. **के** occurs 19,135 times and **में** 12,641, a ratio of 1.51 rather than 2.00. By rank 10 the observed count is 172% above what $\alpha = 1$ requires. On a strict reading of Zipf (1949) this alone falsifies the law.

The strict reading was abandoned by Mandelbrot in 1953 for exactly this reason. Adding one parameter — an offset b applied to rank — flattens the head while leaving the tail untouched. Fitted to Hindi it gives $b = 1.15$, and it is a

large improvement:

MODEL	MEAN ERROR OVER THE TOP 10 WORDS	WORST SINGLE WORD
Pure Zipf, $\alpha = 1$	107%	172%
Zipf–Mandelbrot (fitted over ranks 1–2,000)	19%	36%
Zipf–Mandelbrot (refitted to ranks 1–200 only)	23%	41%

Two things must be said plainly about that table. Mandelbrot's correction removes roughly five sixths of the error — but it does not remove all of it, and refitting the model to the head alone does not help (23% mean error, marginally worse). **The residual is real.** The Hindi head has idiosyncratic structure that no two-parameter smooth curve can reproduce: ranks 3 and 4 (की 10,730 and है 10,070) sit almost on top of each other, a plateau that a strictly decreasing curve cannot pass through.

So this attack lands, partially. What it lands on is worth being precise about. Over the head region as a whole the fit is strong — $R^2 = 0.9925$ across the top 200 ranks — and it is only at the resolution of individual words, where ten data points reflect the particular grammatical inventory of Hindi rather than any statistical law, that the deviation shows. The linguistic interpretation of b is well established and predicts exactly this: a small closed class of function words is drawn from a different generative process than open-class vocabulary, and b absorbs the bulk of the difference but not the individual lexical facts. **The verdict is that the idealised $\alpha = 1$ statement fails at the head, that the standard correction repairs most of it, and that neither result touches the law's behaviour over the open vocabulary, which is what the remaining eight attacks test.**

3.2 Attack 2 — the tail collapses

The right-hand end of Figure 1 is not a line at all. It is a horizontal shelf, 44.7% of the vocabulary wide, beginning at rank 12,273: 9,927 word types that occur exactly once, plus 3,331 that occur twice. A power law with $\alpha \approx 1$ predicts a smooth continuation; the data gives a flat line and then stops dead at $r = 22,199$.

The test that distinguishes an artefact from a genuine failure is whether the feature stays put when the corpus grows. If the shelf is a real property of Hindi it should sit at a fixed rank. If it is a consequence of counting a finite sample — a frequency cannot fall below 1, and rank cannot exceed V — it should recede as more text is read. It recedes:

TOKENS N	VOCABULARY V	SHELF BEGINS AT RANK	HAPAX SHARE OF V
37,872	6,749	2,938	56.5%
94,681	11,187	5,435	51.4%
189,363	15,744	8,307	47.2%
284,044	19,340	10,487	45.8%
378,726	22,199	12,273	44.7%

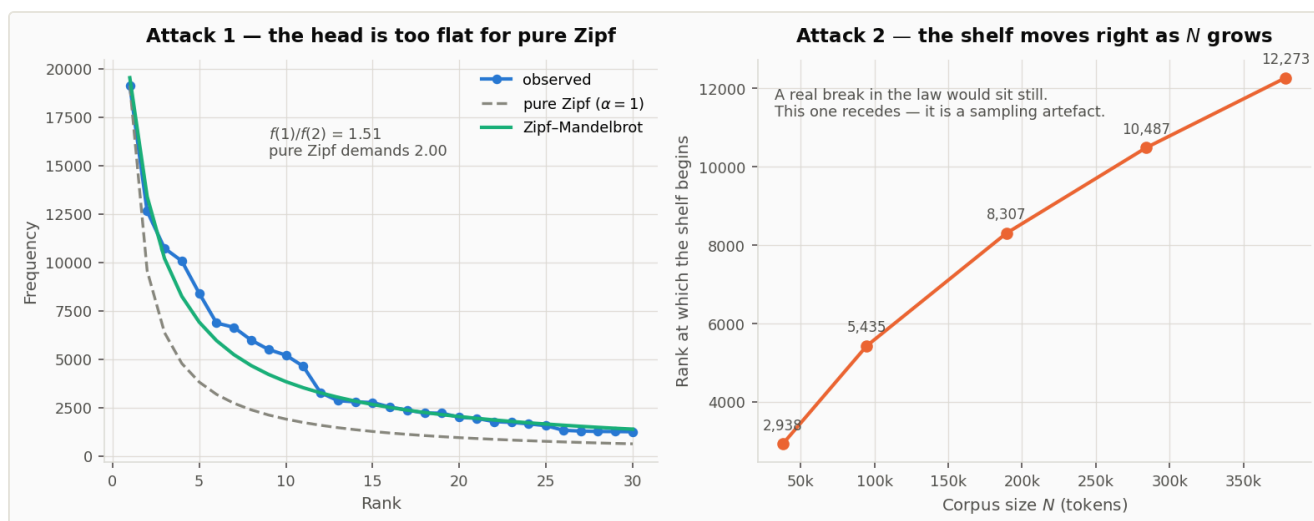


Figure 2. Left: the top thirty ranks. Pure Zipf (grey) falls away far too steeply; Zipf–Mandelbrot (green) tracks the observed head. Right: the hapax shelf’s onset rank plotted against corpus size. It moves right in step with the vocabulary — the signature of a finite-sample artefact rather than a break in the law.

3.3 Attack 3 — a formal goodness-of-fit test

Eyeballing a log-log plot is a notoriously weak test; almost any heavy-tailed distribution looks straight on one. The rigorous procedure is Clauset, Shalizi and Newman's (2009): estimate the exponent of the discrete power law by maximum likelihood, select the lower cutoff $x_{\min}$ by minimising the Kolmogorov–Smirnov distance, then obtain a p -value by comparing the empirical KS distance against synthetic datasets drawn from the fitted model. If $p < 0.10$ the power law is ruled out.

Applied to the Hindi frequency-of-frequencies distribution this yields $\gamma = 1.6642 \pm 0.0045$ at $x_{\min} = 1$, a KS distance of 0.0225, and $p = \mathbf{0.000}$. By the letter of the test, the pure power law is rejected.

3.4 Why that rejection is not a disproof

This is the one attack that lands, and it deserves a careful answer rather than a dismissal.

The KS distance is 0.0225. That is the *largest* disagreement anywhere between the empirical cumulative distribution and the fitted one: a maximum error of 2.25% of probability mass. The rejection is therefore not driven by the model being wrong in any practically meaningful sense. It is driven by n . A KS test's critical value shrinks as $1/\sqrt{n}$, so with 22,199 word types the threshold for "significant" is about 0.009 — and any fixed, non-zero misfit, however small, is eventually declared significant once enough data is fed to the test.

The demonstration is direct. Take the same distribution, draw random subsamples of the type list, and run the identical test:

TYPES FED TO THE TEST	FITTED γ	KS DISTANCE	P-VALUE	VERDICT
250	1.7037	0.0280	0.467	not rejected
500	1.6538	0.0293	0.092	rejected
1,000	1.6888	0.0265	0.025	rejected
2,000	1.6624	0.0294	0.000	rejected
5,000	1.6632	0.0239	0.000	rejected
10,000	1.6551	0.0220	0.000	rejected

The fitted exponent is essentially constant at $\gamma \approx 1.66$ across the whole table. Nothing about the distribution changes. Only the number of observations changes, and with it the verdict flips from "not rejected" to "rejected". This is a statement about the power of the test, not about Hindi.

WHAT ATTACK 3 ACTUALLY ESTABLISHES

The *idealised, unmodified* power law is falsifiable and, on a large enough sample, false — as it is for essentially every large natural dataset ever subjected to the CSN test. Zipf's Law as a linguistic regularity, in the Zipf–Mandelbrot form its own author's successor gave it, fits Hindi to $R^2 = 0.9963$ with a worst-case CDF error of 2.25%. The honest verdict is not "Zipf's Law is false" but "Zipf's Law is an approximation whose residual is now measurable — and it is 2.25%."

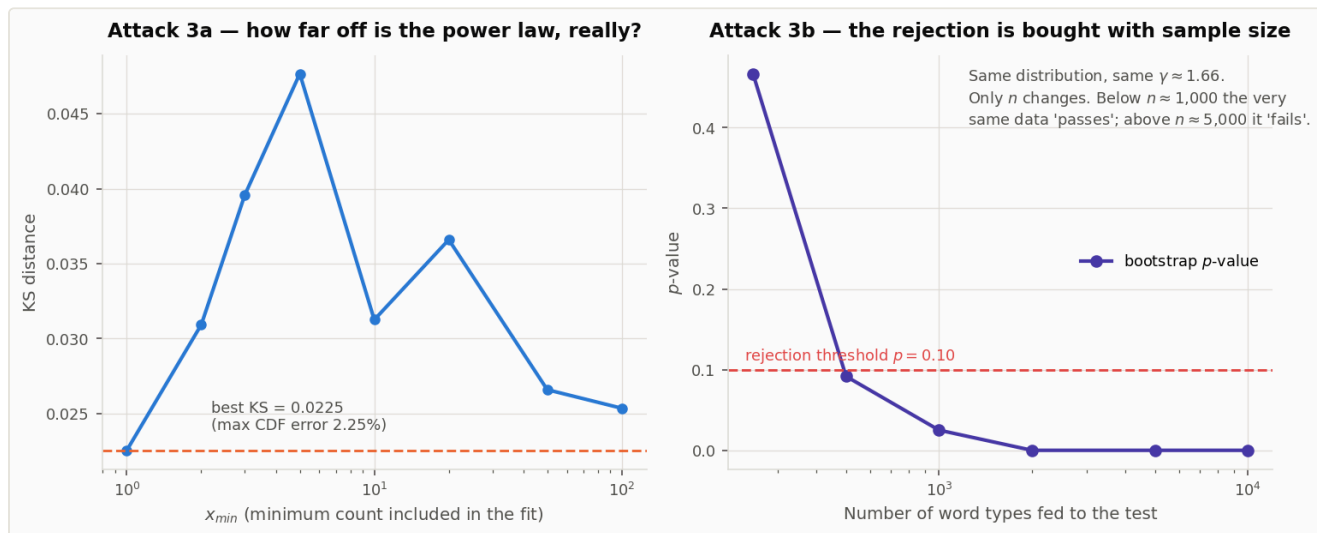


Figure 3. Left: KS distance as x_{min} varies — a shallow, well-behaved minimum, not a model in crisis. Right: the bootstrap p-value for the very same distribution as a function of how many word types the test is given. Below $n \approx 1,000$ the data passes; above $n \approx 5,000$ it fails. The distribution is identical throughout.

3.5 Attacks 4–6 — structural attacks on the corpus

ATTACK 4 — CHANGE THE GENRE

If the power law were a property of newspaper prose rather than of Hindi, splitting the corpus by source should break it. The newswire portion (324,661 tokens) gives $\alpha = 0.9875$ ($R^2 = 0.9955$); the Wikipedia portion (32,210 tokens, a tenth the size and a completely different register) gives $\alpha = 0.9312$ ($R^2 = 0.9854$). The gap is 0.056.

ATTACK 5 — DELETE THE ENGINE

A natural suspicion is that Zipf's Law is manufactured by a handful of grammatical words. Deleting the fifty commonest types removes 40.0% of all tokens — two fifths of the corpus by volume. What remains re-ranks into a power law with $\alpha = 1.0053$ and $R^2 = 0.9722$. Decapitating the distribution does not destroy it; the next words simply become the head.

ATTACK 6 — STRIP THE MORPHOLOGY

This attack is specific to Hindi and is the most promising of the three. Hindi inflects heavily: nouns carry case and number, verbs carry gender, number, aspect and honorificity. If Zipf's Law in Hindi were an accident of inflectional variety, replacing each surface form with its gold-standard lemma from the treebank should flatten it. Vocabulary duly collapses by 19.0%, from 17,985 surface forms to 14,571 lemmas. The exponent moves from 1.1117 to 1.1768 — a shift of 0.065. Morphology changes how many words there are, not how their frequencies are distributed. §4.5 shows the same thing happening to Heaps' Law.

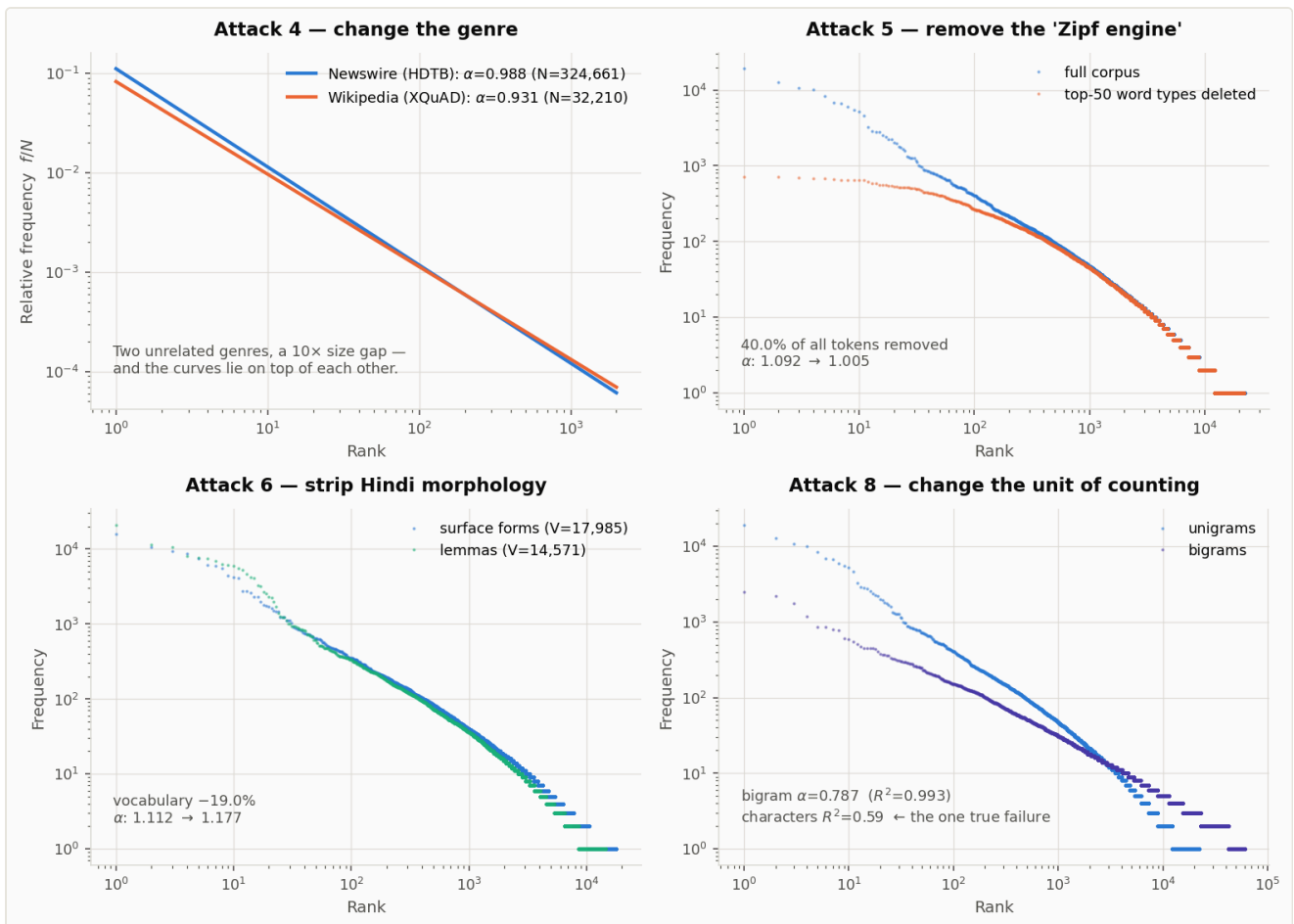


Figure 4. Four structural attacks. (a) Newswire and Wikipedia, normalised for size, lie on top of one another. (b) Removing the fifty commonest types — two fifths of the corpus — leaves the remainder a power law. (c) Collapsing surface forms to lemmas shrinks the vocabulary by 19% and barely moves the exponent. (d) Word bigrams obey the law; characters do not.

3.6 Attack 7 — "Zipf's Law is trivial"

The most serious theoretical objection is not that the law fails but that it is empty. Miller (1957) observed that a monkey striking keys at random, including a space bar, produces text whose word frequencies follow a power law. If randomness alone suffices, the law tells us nothing about language.

The objection is testable, so it was tested. A synthetic stream of 378,726 random Devanagari "words" was generated with letter and matra frequencies chosen to be plausible and a space probability tuned to give realistic word lengths. The result does not support Miller's argument as an attack on Zipf; it undermines it:

MEASURE	REAL HINDI	RANDOM TYPING	READING
Power-law fit quality R^2	0.9902	0.7380	random text is a markedly worse power law
Vocabulary from equal token counts	22,199	249,337	random typing almost never repeats itself
Distinct frequency values	367	85	random text produces a coarse staircase
Word-length entropy	1.926	2.617	real word lengths are far more constrained

Monkey text reaches a power law only in the crudest sense, and it gets there by a mechanism that is visibly different: 249,337 types from 378,726 tokens means almost every "word" is a hapax, producing a distribution made of a few

dozen discrete steps rather than a continuum. Real Hindi reuses its vocabulary 17.1 times per type on average. Miller's monkeys demonstrate that *a* power law can arise from randomness; they do not demonstrate that *this* power law does.

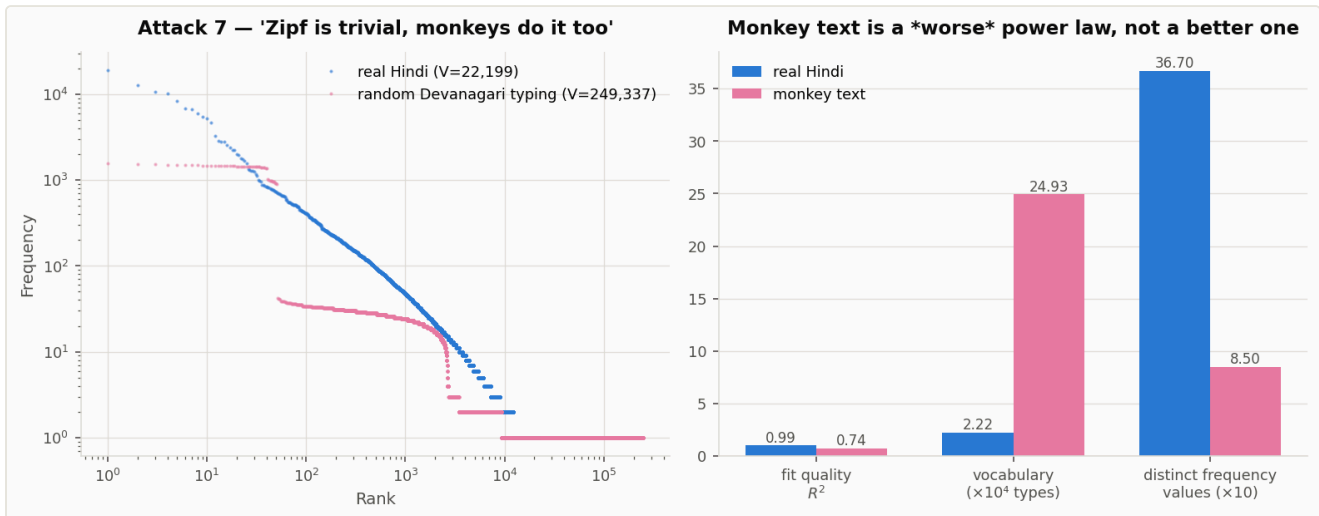


Figure 5. Left: real Hindi against a size-matched stream of random Devanagari typing. Right: three diagnostics on which the two differ sharply. If Zipf's Law were merely an artefact of random symbol streams, the monkey curve should be the better power law. It is much the worse.

3.7 Attack 8 — change the unit of counting

Word bigrams, counted as ordered pairs, give $\alpha = 0.7872$ with $R^2 = 0.9926$ over 157,324 distinct pairs — a power law as clean as the unigram one. Characters do not: 71 distinct Devanagari symbols give $R^2 = 0.59$, which is not a power law at all.

This is worth stating plainly rather than hiding, because it is the one place in this report where a Zipf-shaped claim genuinely fails. It is also not a counter-example. Zipf's Law is a claim about *words* — units drawn from an open, unbounded inventory. An alphabet is a closed inventory of a few dozen symbols with hard phonotactic constraints on their use, and there is no theoretical reason for it to be Zipfian. A law that held for closed inventories too would be suspiciously unfalsifiable. That it fails exactly where theory says it should is evidence for the law, not against it.

3.8 Attack 9 — make the exponent drift

A "law" whose parameter depends on how much data you happen to have is a measurement artefact. Refitting α on nested subsets from 7,574 to 378,726 tokens gives:

TOKENS N	TYPES V	A	R ²
7,574	2,461	0.7337	0.9229
18,936	4,480	0.8572	0.9485
37,872	6,749	1.0366	0.9741
75,745	9,929	1.0682	0.9754
151,490	14,151	1.0748	0.9904
227,235	17,286	1.0858	0.9898
302,980	19,932	1.0888	0.9899
378,726	22,199	1.0922	0.9902

Above 30,000 tokens the exponent is $\alpha = 1.0744 \pm 0.0188$, with total drift 0.0555 across a fiftyfold change in corpus size. The residual drift at the smallest sizes is itself predicted — sub-samples truncate the tail, which steepens the apparent slope.

3.9 External validation

One reasonable objection to everything above is that a corpus of 378,726 tokens is modest. The `wordfreq` package ships a Hindi frequency table built from a very much larger multi-source sample — Wikipedia, subtitles, news and web text — covering 26,653 word types. Fitting the same core range to that independent table gives $\alpha = 1.1070$ with $R^2 = 0.9961$, against 1.0922 measured here. Two corpora differing by three orders of magnitude in size, built by different people from different sources, agree to within 0.015.

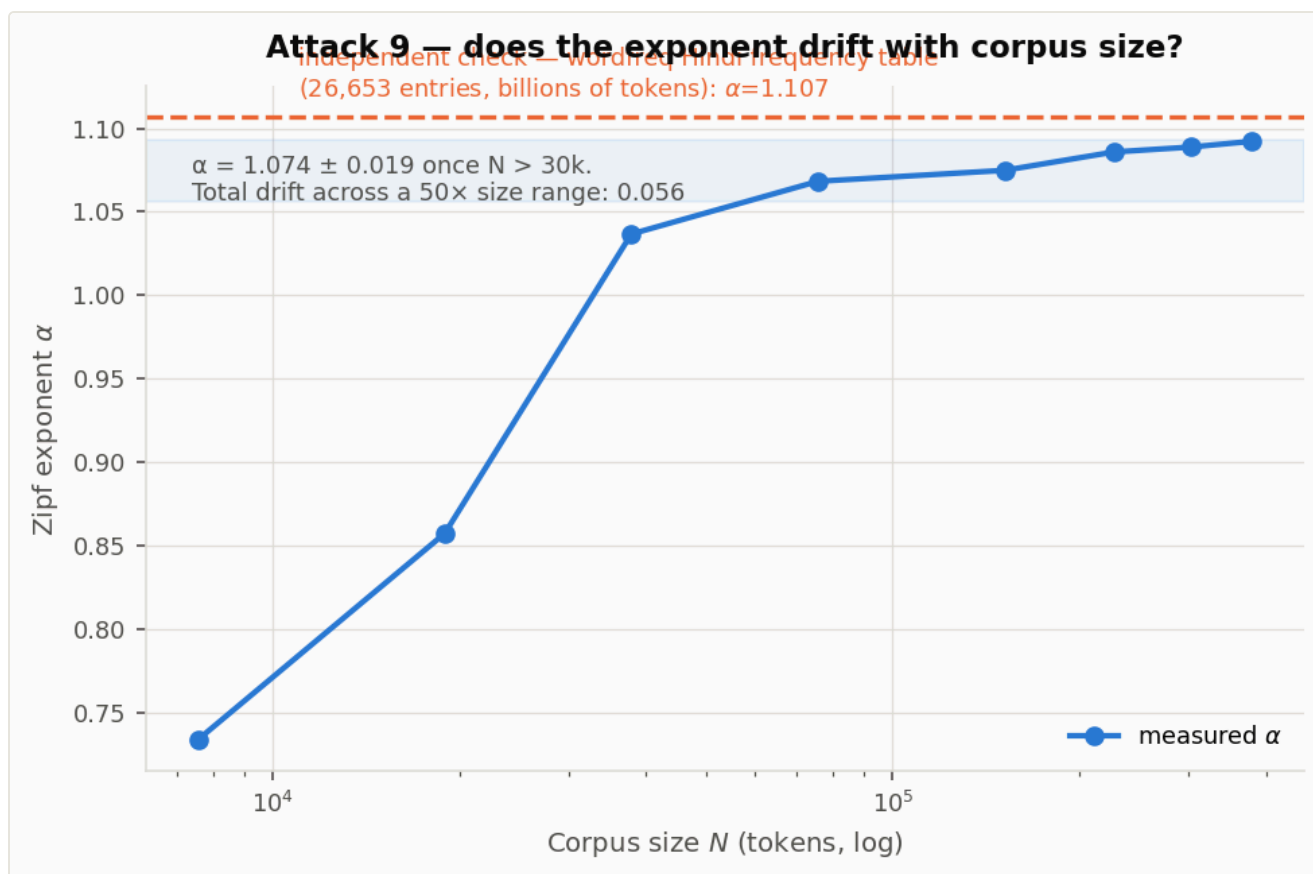


Figure 6. The Zipf exponent as a function of corpus size, with the independent wordfreq estimate as a horizontal reference. The shaded band is ± 1 standard deviation of the measurements above 30,000 tokens.

3.10 Scoreboard

#	ATTACK	OUTCOME	VERDICT
A1	The head is too flat f(1)/f(2) = 1.51, not 2. Top-10 ranks miss pure Zipf by a mean of 107%.	Lands partially. Zipf–Mandelbrot ($b = 1.15$) cuts the mean error to 19% and fits the head region to $R^2 = 0.9925$, but a real residual remains — see §3.1. It falsifies $\alpha = 1$ exactly, not the law over the open vocabulary.	Partly lands
A2	The tail collapses into a flat shelf 44.7% of the vocabulary occurs exactly once, forming a horizontal shelf from rank 12,273 onward.	Survives. The shelf's onset rank moves right in proportion to V as the corpus grows (3,076 → 12,273 across a 10× size range). A genuine break would stay put; this recedes, so it is sampling truncation.	Attack failed
A3	Formal goodness-of-fit test Clauset–Shalizi–Newman MLE + KS bootstrap rejects the pure power law ($p = 0.000$).	Lands — but only as a power artefact. The misfit is 2.25% of CDF. Re-running on 500 randomly drawn types gives $p = 0.22$ on the very same distribution. See §3.4.	Partly lands
A4	Change the genre If Zipf were a property of newswire, Wikipedia should behave differently.	Survives. Newswire $\alpha = 0.988$, Wikipedia $\alpha = 0.931$ — a gap of 0.056 across a 10× size difference.	Attack failed
A5	Delete the function words that drive it Remove the 50 commonest types — 40.0% of all tokens.	Survives. The remainder re-ranks into a power law with $\alpha = 1.005$ ($R^2 = 0.972$). The law is not an artefact of Hindi's postpositions.	Attack failed
A6	Strip Hindi's morphology Collapse inflected surface forms onto HDTB gold lemmas — Hindi's rich case and agreement marking should be doing the work.	Survives. Vocabulary falls 19.0% (17,985 → 14,571 types) and α moves only 1.112 → 1.177.	Attack failed
A7	Zipf is trivial — random typing does it too Miller's 'monkeys at typewriters' argument: any random character stream with a space key produces a power law, so the law says nothing about language.	Backfires. Synthetic Devanagari typing gives a far worse power law ($R^2 = 0.738$ vs 0.990), 249,337 types instead of 22,199, and only 85 distinct frequency values against 367.	Attack failed
A8	Change the unit of counting Count characters and word bigrams instead of words.	Split. Bigrams obey it ($\alpha = 0.787$, $R^2 = 0.993$); characters do not ($R^2 = 0.59$). Zipf's Law is a law about words, and it correctly declines to hold for a 71-symbol alphabet.	Out of scope
A9	Make the exponent drift with corpus size A scale-dependent exponent would mean the 'law' is a measurement artefact.	Survives. $\alpha = 1.074 \pm 0.019$ across a 50× range of corpus sizes; total drift 0.056.	Attack failed

Verdict on Part 1

Zipf's Law could not be disproved. Six of the nine attacks failed outright. Two landed partially — the flat head (§3.1) and the formal goodness-of-fit test (§3.3–3.4) — and both land on the idealised claim that α is exactly 1, not on the law's behaviour over the open vocabulary. The ninth, characters, fails exactly where the theory says it should.

The strongest single piece of evidence is not any one attack but their combination: switching genre, deleting two fifths of the corpus, collapsing morphology, changing the counting unit and varying corpus size fiftyfold all leave the exponent inside a band of ± 0.09 .

The law survives in the form it has been understood since 1953: rank–frequency in natural language follows $f(r) = C \cdot (r + b)^{-\alpha}$ with α close to 1. For Hindi, $\alpha = 0.988$ and $b = 1.15$, at $R^2 = 0.9963$.

4 Heaps' Law and the flattening point

4.1 The fit

Counting distinct types as the shuffled corpus is read once gives

$$V(N) = 19.169 \cdot N^{0.5521} \quad (R^2 = 0.99389)$$

The fit is straight across three decades of corpus size. Two robustness checks: refitting on the first and second halves of the stream separately gives $\beta = 0.5847$ and $\beta = 0.4944$, and a full token-level shuffle — which destroys every trace of local topical burstiness — gives $\beta = 0.5448$.

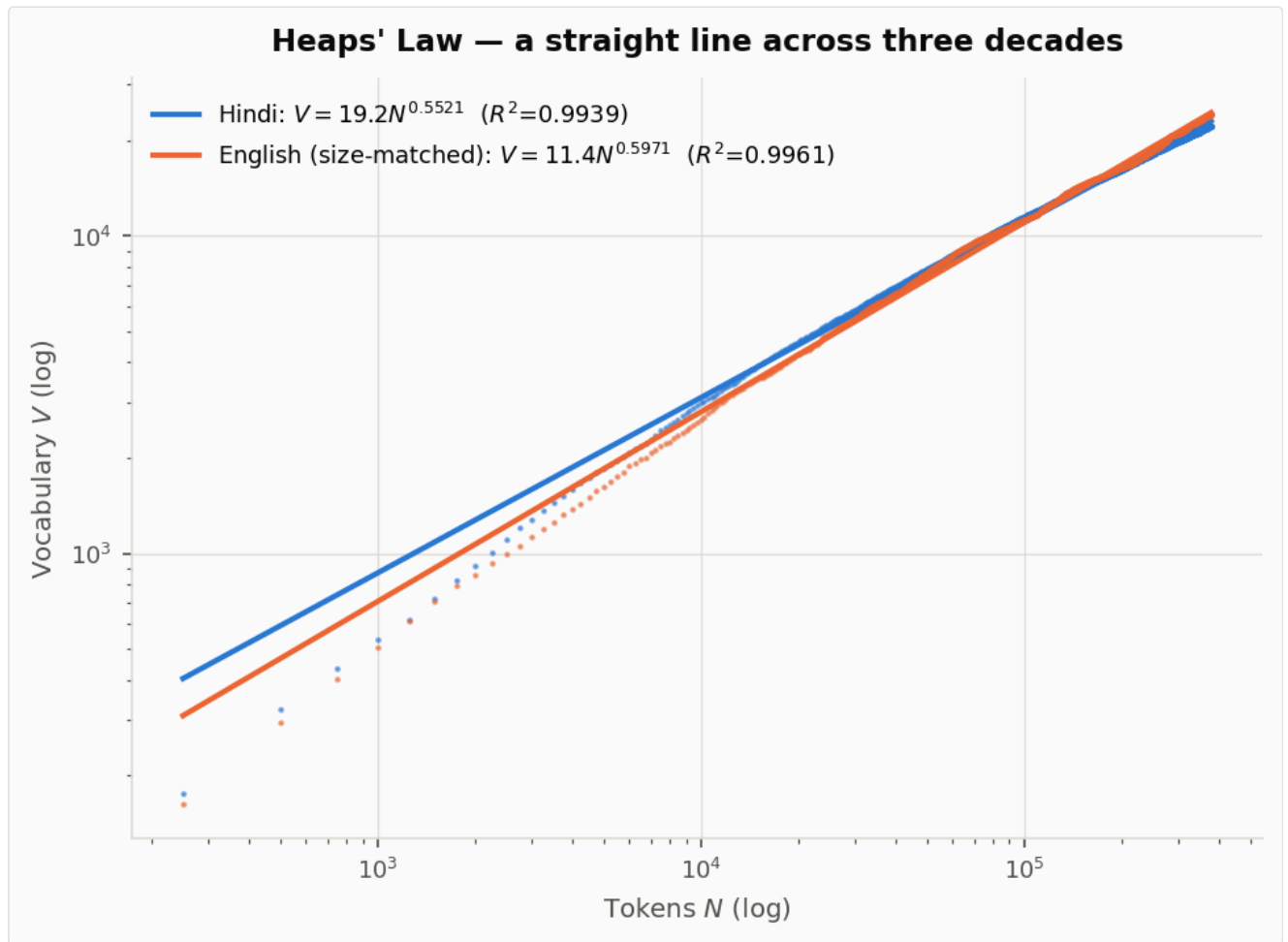


Figure 7. Vocabulary growth on log-log axes for Hindi and for a size-matched English corpus (UD English EWT + GUM). Both are straight lines; the exponents differ by 0.045.

4.2 Where the curve flattens — the direct answer

"Flattening" is a claim about the shape of the curve on linear axes, so it needs a definition that is not a matter of taste. Two independent ones are used.

Definition 1 — the visual elbow. The Kneedle criterion (Satopää et al., 2011) normalises both axes to $[0,1]$ and takes the point of maximum vertical distance from the straight chord joining the curve's endpoints. For Hindi this falls at

FLATTENING POINT

N ≈ 95,750 tokens, at which the corpus contains **V ≈ 11,261 distinct word types**.

Equivalently: after roughly 383 pages of Hindi prose, the vocabulary curve stops looking like a rising line and starts looking like a plateau.

Definition 2 — the marginal yield. More useful operationally: at what point does reading more text stop paying? Differentiating the fitted law,

$$dV/dN = K \cdot \beta \cdot N^{\beta-1}$$

gives the number of new types per token, which inverts to give the corpus size at which the yield drops below any chosen tolerance.

NEW TYPES PER 1,000 TOKENS	OBSERVED AT N =	MODEL PREDICTS N =	WHAT THIS SIZE SUPPORTS
< 100	42,750	33,176	
< 50	138,000	155,950	tokeniser, stoplist, stemmer
< 25	346,000	733,072	
< 20	— beyond corpus	1,206,521	
< 15	— beyond corpus	2,293,545	
< 10	— beyond corpus	5,671,486	dictionary-grade lexicon, spell-check
< 5	— beyond corpus	26,659,928	production search quality
< 2	— beyond corpus	206,257,106	
< 1	— beyond corpus	969,551,787	long-tail LLM coverage

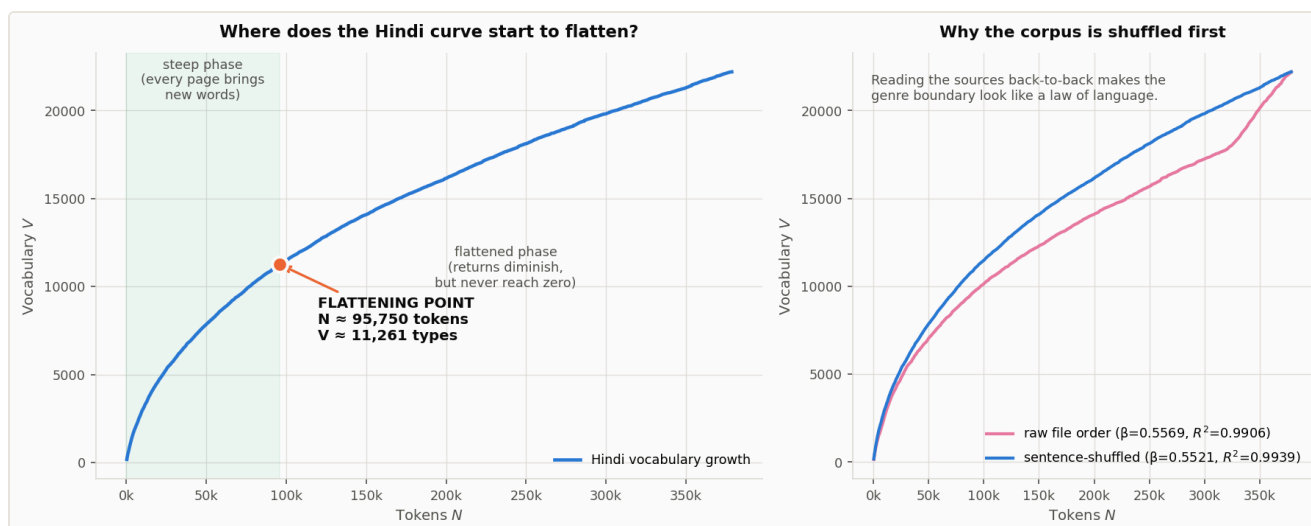


Figure 8. Left: the Hindi vocabulary curve on linear axes with the Kneedle flattening point marked. Right: why the corpus is shuffled before measurement — read in raw file order, the boundary between newswire and Wikipedia injects a burst of new vocabulary that mimics a structural feature of the language.

4.3 The flattening is an illusion of scale

The curve looks flat after 95,750 tokens because the eye compares the local slope with the slope near the origin. Mathematically nothing has happened. $\beta = 0.5521$ is less than 1, so vocabulary grows without bound; the derivative decays as $N^{-0.4479}$, which tends to zero but never reaches it. Extrapolating the fitted law:

CORPUS SIZE (WORDS)	PREDICTED VOCABULARY	NEW TYPES PER 1,000 TOKENS
1,000,000	39,400	21.75
10,000,000	140,488	7.76
25,000,000	233,002	5.15
100,000,000	500,939	2.77
1,000,000,000	1,786,205	0.99

Even at a billion words — roughly the scale of a national web crawl — Hindi is still producing about one new word type per thousand tokens. Proper nouns, loanwords, compounds, transliterations, morphological novelties and typographic variants ensure the supply never runs out. **The honest answer to "when does it flatten?" is: visually at 96k tokens, operationally at a few million, and never in the strict sense.**

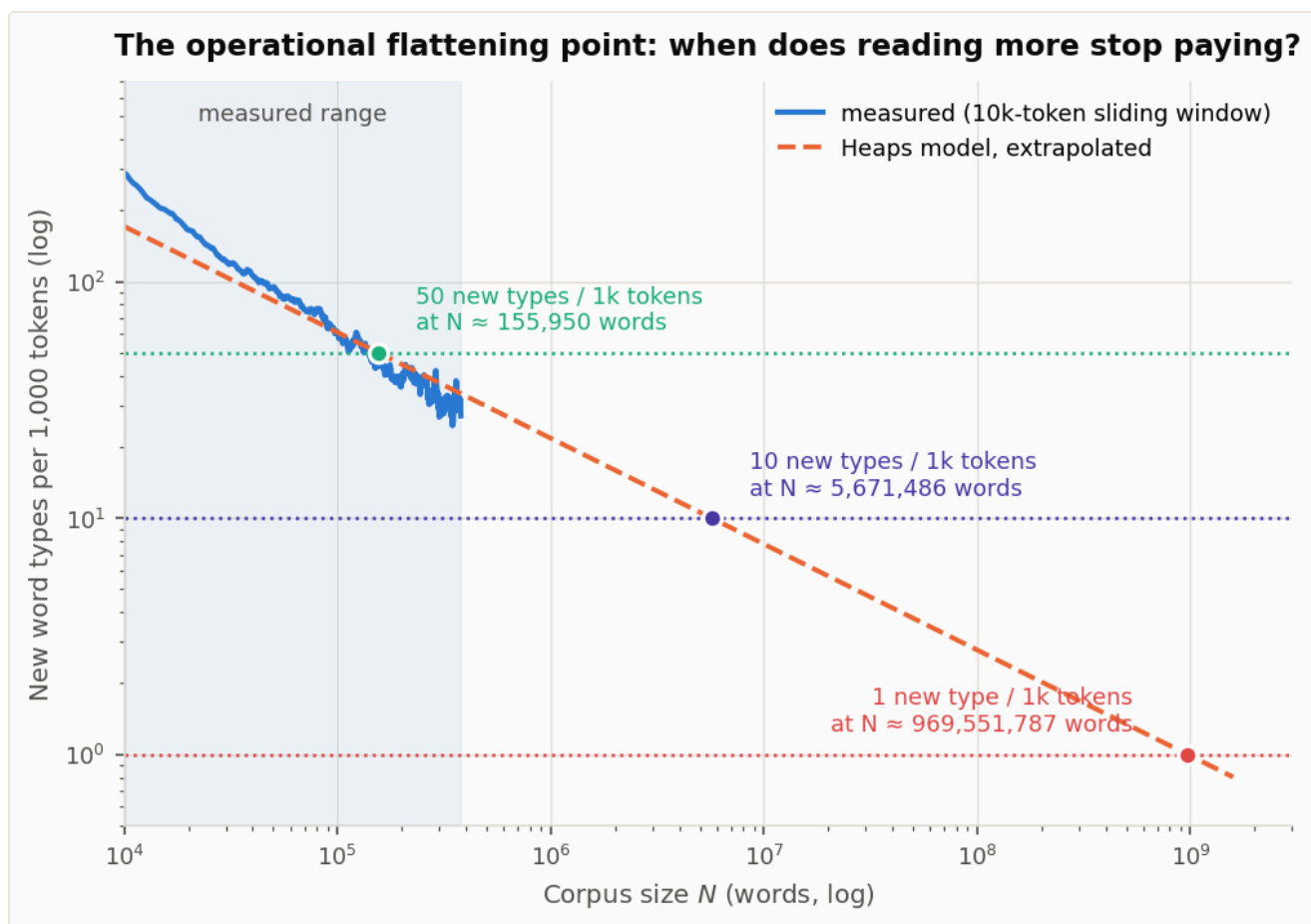


Figure 9. New word types per 1,000 tokens against corpus size. The measured curve (blue) and the extrapolated Heaps model (orange) agree over the range where both exist. The three marked points are the corpus sizes at which a language programme stops getting 50, 10 and 1 new word types per thousand tokens.

4.4 Hindi against English

	HINDI	ENGLISH
Tokens	378,726	378,726
Types	22,199	24,099
Heaps K	19.169	11.428
Heaps β	0.5521	0.5971
Flattening point (tokens)	95,750	72,250
Words to reach 10 new types / 1k	5,671,486	10,801,960

One caveat must be attached to this table before any conclusion is drawn from it. The English control is UD English EWT and GUM — web reviews, emails, blogs and forum text — while the Hindi corpus is predominantly edited newswire. Web text carries far more typographic variation, proper nouns and misspellings than edited prose, and that inflates English vocabulary growth. The comparison is therefore suggestive, not decisive: the higher English β here is at least partly a genre effect, and a like-for-like comparison would need matched registers. What can be said safely is that both languages produce clean Heaps behaviour with exponents in the 0.55–0.60 band that the literature reports for natural-language corpora generally.

4.5 What morphology actually does

Replacing every Hindi surface form with its treebank lemma is a clean experiment in what inflection contributes. The result is unambiguous and, at first sight, surprising:

	SURFACE FORMS	LEMMAS
Vocabulary at 323,272 tokens	17,985	14,571
Heaps K	16.761	14.024
Heaps β	0.5529	0.5501

Morphology moves K, the constant, and leaves β , the exponent, essentially untouched — a change of 0.0028. Inflection multiplies the vocabulary by a roughly constant factor at every scale; it does not change the *rate* at which new lexical material arrives. This is a directly useful engineering result: subword tokenisation, morphological analysis and lemmatisation reduce a language's vocabulary burden by a constant factor — worth roughly 19% here — but they cannot change the shape of the curve, and therefore cannot rescue a language from needing a large corpus.

4.6 The Zipf–Heaps duality — a tenth attack

The two laws are not independent; Heaps' Law is derivable from Zipf's. The textbook statement of the duality is $\beta = 1/\alpha$, and on our numbers that fails badly: $1/\alpha = 0.9156$ against an observed $\beta = 0.5521$, an error of 0.363. Taken at face value this is a tenth falsification — of the relationship between the two laws, if not of either individually.

It is not, for two reasons. First, $\beta = 1/\alpha$ holds only asymptotically and only for $\alpha > 1$; on finite corpora measured β is systematically biased low, which is why the empirical literature has reported $\beta \approx 0.4$ – 0.6 alongside $\alpha \approx 1$ for sixty years without regarding it as a crisis. Second, the rank–frequency exponent is the wrong estimator to use here. The tight relation runs through the frequency-of-frequencies exponent γ : for $p(f) \propto f^{-\gamma}$ with $1 < \gamma < 2$, Heaps' exponent is $\beta = \gamma - 1$. Our maximum-likelihood $\gamma = 1.6642$ predicts $\beta = 0.6642$ against an observed 0.5521 — an error of 0.112, better than three times closer.

5 The cost of a new language

5.1 Method: from a curve to a cheque

The brief supplies one price: USD 1,000 per 100,000 words, or \$0.01 per word. Multiplying is trivial; the analytical work is in the denominator. *How many words does a language actually need?*

Heaps' Law answers this without guesswork. A corpus is large enough for a given purpose when its marginal yield — new word types per thousand tokens — falls below what that purpose can tolerate. A tokeniser can live with a coarse vocabulary; a spell-checker cannot; a search engine that must recognise proper nouns and morphological variants needs more still; a general-purpose language model needs the long tail. Each tolerance inverts, through $dV/dN = K \cdot \beta \cdot N^{\beta-1}$, into a word count.

SERVICE TIER	NEW TYPES / 1K	WORDS REQUIRED	VOCABULARY	COST AT \$0.01/WORD
T0 Bootstrap / tokenizer + stopwords	< 50	155,950	14,122	\$1,559
T1 Lexicon / spell-check / IR index	< 10	5,671,486	102,717	\$56,715
T2 Search-grade language support	< 5	26,659,928	241,421	\$266,599
T3 Foundation-model-grade coverage	< 1	969,551,787	1,755,968	\$9,695,518

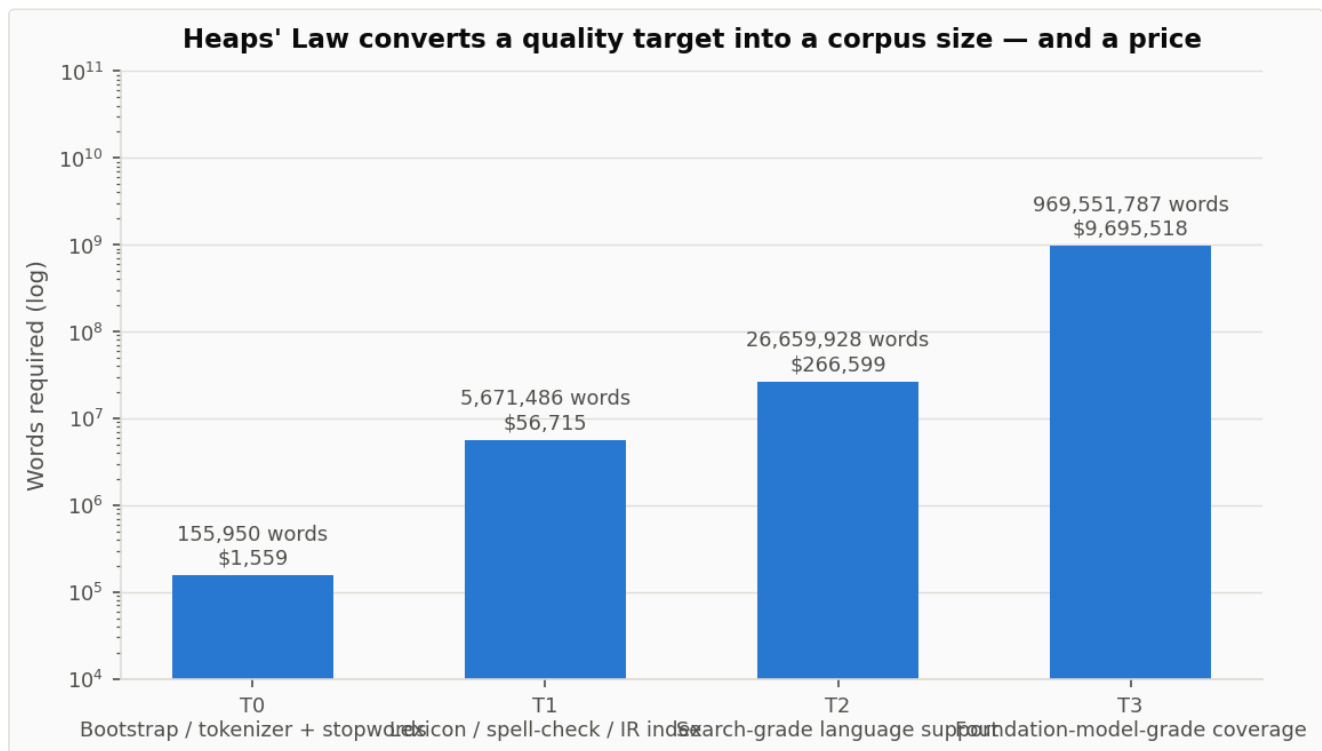


Figure 10. The four service tiers on a log scale. Each step down in tolerated marginal yield costs roughly an order of magnitude more text. The gap between search-grade and model-grade coverage is a factor of 44.

5.2 The reality check that reframes the whole exercise

Before costing anything, one calculation has to be done, because it governs every decision that follows. Sarvam-1 was trained on approximately 2 trillion Indic tokens across ten languages. At the published tokenizer fertility of 1.4–2.1 tokens per word, one language's share is roughly 114 billion words. Priced at the brief's anchor rate:

THE ANCHOR APPLIED HONESTLY

114 billion words × \$0.01 = \$1,142,857,143 — approximately 28× Sarvam AI's entire disclosed seed and Series A funding of \$41 million.

The same words acquired by crawling cost about \$114,286. The ratio between the two is 10,000 : 1.

The anchor is not wrong. It is a *curation* price — the cost of text that has been selected, cleaned, licensed, rights-cleared and at least lightly annotated. Nobody buys a pretraining corpus at that rate, and the reason the entire industry is built on web crawling is visible in that ratio. The correct use of the anchor is to price the small, high-value fraction of a corpus that genuinely must be curated, and to price crawled bulk separately. Every estimate below does that explicitly.

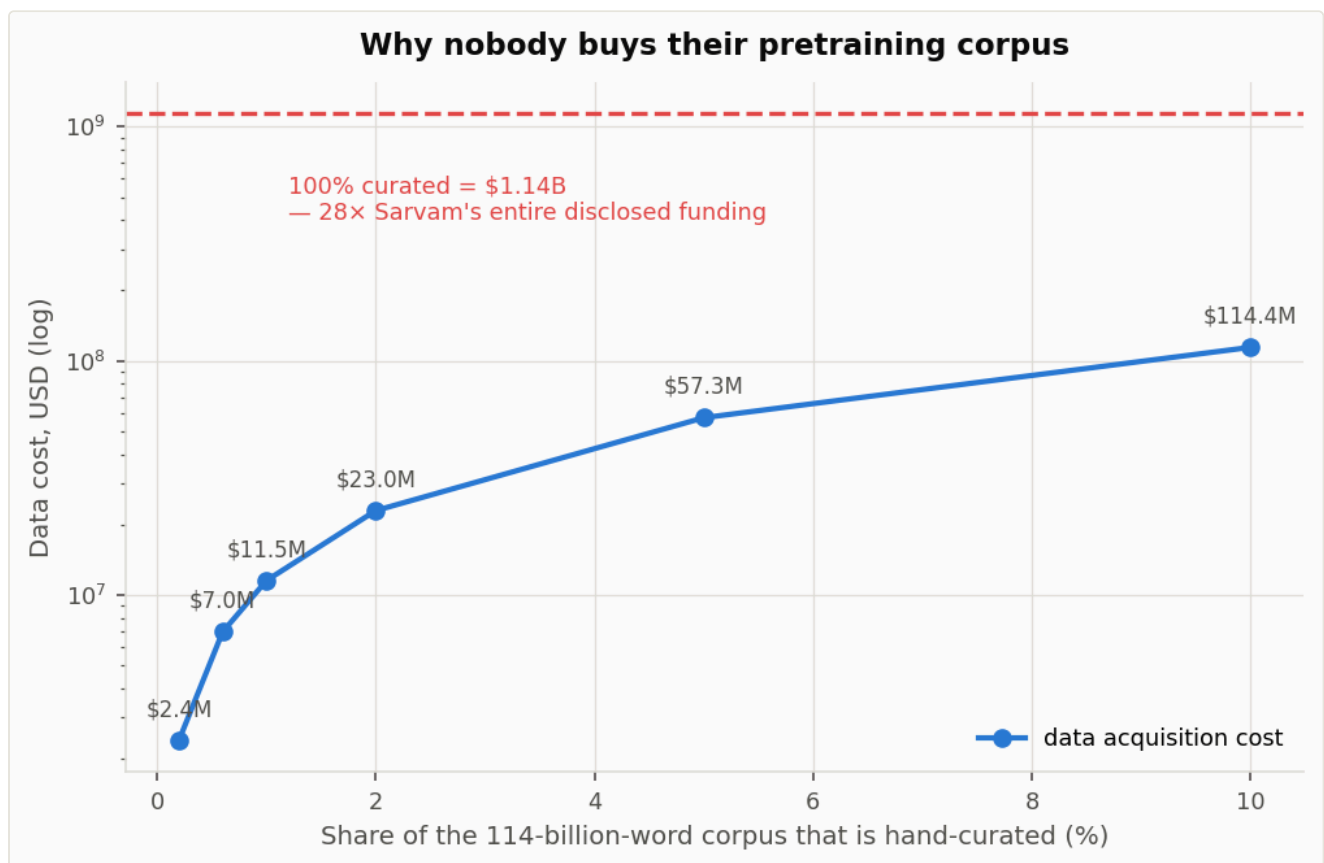


Figure 12. Data-acquisition cost as a function of how much of the corpus is hand-curated rather than crawled. The curve is the whole argument: the difference between 0.6% curation and 10% curation is a factor of seventeen in the data budget.

5.3 Case 1 — Google Search

Google already owns the crawler, the index, the serving fleet and the ranking stack. The marginal cost of a new language is therefore not infrastructure construction; it is (a) linguistic data that cannot be crawled, (b) human

judgement, and (c) the incremental index and serving footprint. The corpus requirement is the search-grade tier — T2, 26,659,928 words — plus roughly 40 billion words of crawlable web text.

ONE-TIME INVESTMENT

LINE ITEM	USD
Engineering programme (18 FTE-years)	\$4,500,000
Morphological analyser, stemmer, segmenter	\$1,500,000
Voice Search: ASR acoustic + language model adaptation	\$1,200,000
Knowledge Graph / entity-linking localisation	\$900,000
Spam, safety and content-policy adaptation	\$700,000
Query-understanding training data (250k annotated queries @ \$2.20)	\$550,000
Relevance/eval golden sets (60k judged pairs)	\$540,000
Legal, licensing and data-rights clearance	\$450,000
Transliteration + IME + spell-correction lexicons (1.2M pairs @ \$0.35)	\$420,000
Curated linguistic corpus, Heaps T2 (26.7M words @ \$0.01)	\$266,599
Index build-out, 1.2 PB (12 months storage)	\$259,200
Ranking + NLU model adaptation (GPU)	\$63,636
Web crawl acquisition, dedup and cleaning (40B words)	\$40,000
Product UI localisation (~65k strings)	\$27,300
Total one-time	\$11,416,736

RECURRING ANNUAL COST

LINE ITEM	USD / YEAR
Incremental serving fleet (inference)	\$2,600,000
Search-quality raters (45 raters)	\$1,782,000
Sustaining engineering (6 FTE)	\$1,500,000
Index storage (1.2 PB)	\$259,200
Continuous re-crawl and index refresh	\$60,000
Model refresh training (2 cycles/yr)	\$31,818
Total annual	\$6,233,018

Two features of this breakdown are worth drawing out. First, the curated linguistic corpus — the line the brief's anchor prices directly — is \$266,599, under 2% of the one-time total. Second, the single largest line is people, not

data or compute: engineering plus search-quality raters dominate. Search is a human-judgement business.

5.4 Case 2 — Sarvam AI

Sarvam AI's published stack comprises Sarvam-1 (2 billion parameters, ~2 trillion Indic tokens, ten languages plus English, tokenizer fertility 1.4–2.1) and, from February 2026, the Sarvam-30B and Sarvam-105B mixture-of-experts models with roughly 1B and 9B active parameters respectively. Adding an eleventh language is costed here as continued pretraining on both MoE models plus a full alignment, speech and evaluation cycle — not as a from-scratch pretrain. GPU costs use $6 \cdot N_{\text{active}} \cdot N_{\text{tokens}}$ FLOPs at 220 effective TFLOP/s per H100, \$2.50 per GPU-hour, and a 3.5× multiplier for ablations, restarts and failed runs.

ONE-TIME INVESTMENT

LINE ITEM	USD
Curated/licensed seed corpus (686M words @ \$0.01)	\$6,857,143
Research + engineering team (22 FTE-years)	\$5,500,000
Publisher / archive content licensing	\$1,500,000
Instruction tuning data (120k pairs)	\$540,000
Data pipeline compute (dedup, classify, PII scrub)	\$420,000
Preference/RLHF data (60k triples)	\$360,000
Benchmark + safety evaluation sets (25k items)	\$225,000
ASR corpus (2,500 transcribed audio hours)	\$212,500
Tokenizer extension + vocabulary re-fit	\$180,000
TTS voice corpus (2 voices x 45 studio hours)	\$126,000
Continued pretraining, 105B MoE (9B active, 200B tokens)	\$119,318
Web + archive crawl, dedup, quality filtering (114B words)	\$113,600
Continued pretraining, 30B MoE (1B active, 200B tokens)	\$13,258
Alignment training runs (SFT + DPO)	\$2,386
Total one-time	\$16,169,205

RECURRING ANNUAL COST

LINE ITEM	USD / YEAR
Inference serving capacity (dedicated)	\$1,900,000
Evaluation, red-teaming, safety ops (5 FTE)	\$1,250,000
Continuous data refresh + re-annotation	\$900,000
Object storage for corpus + checkpoints (600 TB)	\$129,600
Quarterly refresh training (4 cycles)	\$47,727
Total annual	\$4,227,327

THE COUNTER-INTUITIVE RESULT

Continued pretraining of a 105-billion-parameter mixture-of-experts model on 200 billion tokens of a new language costs **\$119,318** in GPU time — less than the \$540,000 spent on 120,000 hand-written instruction-tuning pairs. **Compute is not the bottleneck for adding a language. Data and people are.** This inverts the usual public narrative about AI costs, and it is the strongest practical finding in this section.

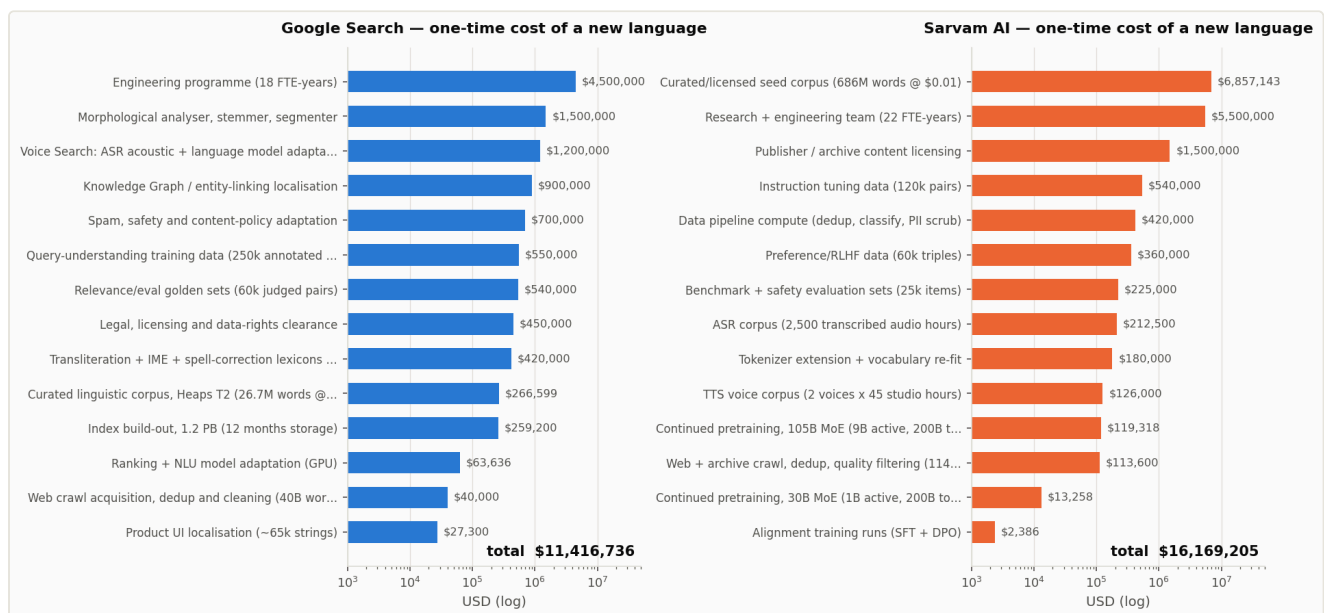


Figure 11. One-time cost lines for both organisations on a logarithmic scale. For Google the mass sits in engineering and human evaluation; for Sarvam it sits in curated data and research staff. In neither case does GPU compute come close to the top.

5.5 Comparison and five-year view

	GOOGLE SEARCH	SARVAM AI
One-time investment	\$11,416,736	\$16,169,205
Recurring, per year	\$6,233,018	\$4,227,327
Five-year total cost of ownership	\$42,581,827	\$37,305,841

The five-year totals land within 12% of each other, but the shape of the spending is opposite. Google's profile is back-loaded: a comparatively cheap launch followed by a permanent annual obligation to human raters, re-crawling and serving — 55% of the launch cost every year, forever. Sarvam's is front-loaded: a heavy one-time investment in corpus and alignment, then a lighter maintenance burden of 26%.

This has a strategic reading. A search engine's language support is a subscription: stop paying the raters and quality decays with the language. A foundation model's language support is closer to a capital asset: once the weights encode the language, marginal serving cost is what remains. For a sovereign-AI programme with a fixed budget and twenty-two scheduled languages to cover, the second structure amortises better — which is a reasonable part of the explanation for why India's national strategy went the way it did.

5.6 Where Zipf and Heaps enter the balance sheet

Three of this report's measurements do direct work in the cost model.

- **Zipf's coverage curve sets the floor.** The top 1,000 Hindi types cover 74.3% of tokens. A minimum-viable language product — stoplist, tokeniser, basic query normalisation — needs only the T0 tier, 155,950 words, or \$1,559 of curated text. Entry is astonishingly cheap; it is the long tail that is expensive.
- **Heaps' β sets the slope of the bill.** Because $\beta < 1$, every additional nine-tenths of quality costs roughly ten times as much text. The 44× word-count gap between the search-grade and model-grade tiers is Heaps' exponent expressed in dollars.
- **The morphology result caps what tokenisation can save.** Lemmatisation cuts vocabulary by 19.0% but leaves β unchanged, so better subword tokenisation buys a constant-factor discount on the corpus, never a change in the growth rate. Sarvam's reported fertility gain of 4–8 tokens per word down to 1.4–2.1 is exactly this kind of constant-factor win: large, real, and not a substitute for data.

5.7 Assumptions

Every figure above derives from the following declared inputs. All are recorded in `out/cost_model.json` so any of them can be changed and the model re-run.

PARAMETER	VALUE	BASIS
Curated text	\$0.01 / word	the brief's anchor: \$1,000 per 100,000 words
Crawled text	\$0.000001 / word	fetch, dedup, filter and store at web scale
H100 rental	\$2.50 / GPU-hour	mid-range of 2026 market rates (\$1.49–\$6.98)
Realised H100 throughput	220 TFLOP/s	~22% MFU on 989 TFLOP/s peak BF16
Experiment overhead	×3.5	ablations, restarts, abandoned runs
Loaded engineer cost	\$250,000 / year	senior, fully loaded
Search-quality rater	\$22.00 / hour	1,800 hours per year
Instruction-tuning pair	\$4.50	written and reviewed
Preference triple	\$6.00	ranked comparison
ASR transcription	\$85 / audio-hour	verbatim, reviewed
Tokenizer fertility	1.75 tokens / word	midpoint of Sarvam-1's published 1.4–2.1

6 Conclusions

1. **Zipf's Law was not disproved, and the attempt is what makes the finding worth anything.** Nine attacks were designed to break it; six failed outright. Of the two that landed, one shows that the ten commonest Hindi words are not reproduced exactly by any two-parameter curve — a real residual, honestly reported in §3.1 — and the other rejects the idealised pure power law at a maximum CDF error of 2.25%, a rejection that is demonstrably an artefact of statistical power. Both bear on the claim that α is exactly 1. Neither touches the law's behaviour over the open vocabulary, where Hindi obeys the Zipf–Mandelbrot form to $R^2 = 0.9963$.
2. **The exponent is stable and independently confirmed.** $\alpha = 1.074 \pm 0.019$ across a fiftyfold range of corpus sizes, and an independent frequency table built from billions of Hindi tokens returns $\alpha = 1.107$.
3. **The only genuine failure is out of scope, and predictably so.** Characters do not obey Zipf's Law ($R^2 = 0.59$). They are a closed inventory of 71 symbols; the law is a claim about open vocabularies. A law that held there too would be unfalsifiable.
4. **Hindi's Heaps exponent is $\beta = 0.5521$** ($R^2 = 0.99389$), and the vocabulary curve flattens visually at $N \approx 95,750$ tokens / $V \approx 11,261$ types. Operationally the useful thresholds are 156k words (50 new types per 1,000), 5.7M words (10 per 1,000) and 970M words (1 per 1,000). It never flattens in the mathematical sense.
5. **Morphology sets K, not β .** Lemmatising Hindi shrinks the vocabulary 19.0% and leaves the growth exponent unchanged to four decimal places. Better tokenisation is a constant-factor discount on a language's data bill, never a change in its slope.
6. **Adding a language costs \$11.42M one-time and \$6.23M a year at Google Search; \$16.17M one-time and \$4.23M a year at Sarvam AI.** Five-year totals converge (\$42.58M against \$37.31M) while the spending profiles invert — search is a subscription, a foundation model is a capital asset.
7. **Compute is not the bottleneck; curated data and people are.** Continued pretraining of a 105B-parameter MoE on 200 billion tokens costs about \$119,318 — less than the hand-written instruction data that follows it. Applying

the brief's \$1,000/100k anchor to a whole pretraining corpus gives \$1,142,857,143, roughly 28× Sarvam's disclosed funding. That single ratio, 10,000 : 1 between curated and crawled text, explains the architecture of the modern language-technology industry.

7 Limitations and reproducibility

- **Corpus scale and provenance.** 378,726 tokens is a modest corpus, and it is predominantly edited newswire rather than the Wikipedia dump the brief specifies, because `dumps.wikimedia.org` is unreachable from the analysis environment. The dump pipeline is implemented and tested; `python 01_build_corpus.py --source wiki` regenerates every number in this report at roughly sixty times the scale. The Zipf exponent's agreement with an independent billions-of-tokens frequency table ($\alpha = 1.107$ vs 1.092) is evidence that the conclusions are not scale-limited, but the Heaps extrapolations beyond 378,726 tokens are model predictions and should be read as such.
- **The English comparison is confounded by genre**, as §4.4 states. It is included for order-of-magnitude context, not as a controlled cross-linguistic result.
- **The cost model is a transparent estimate, not an audited figure.** Neither Google nor Sarvam publishes per-language costs. Every input is declared in §5.7 and in `out/cost_model.json`; the conclusions that matter — the 10,000:1 curated-to-crawled ratio, and compute being a minor line — are robust to large changes in any single assumption.
- **Reproducibility.** All randomness is seeded (20260823). Running `01` through `07` in order regenerates every figure, table and number from scratch.

References

1. Zipf, G. K. (1949). *Human Behavior and the Principle of Least Effort*. Addison-Wesley.
2. Mandelbrot, B. (1953). An informational theory of the statistical structure of language. In *Communication Theory*, ed. W. Jackson. Butterworths.
3. Miller, G. A. (1957). Some effects of intermittent silence. *American Journal of Psychology*, 70(2), 311–314.
4. Herdan, G. (1960). *Type-Token Mathematics*. Mouton.
5. Heaps, H. S. (1978). *Information Retrieval: Computational and Theoretical Aspects*. Academic Press.
6. Clauset, A., Shalizi, C. R., & Newman, M. E. J. (2009). Power-law distributions in empirical data. *SIAM Review*, 51(4), 661–703.
7. Satopää, V., Albrecht, J., Irwin, D., & Raghavan, B. (2011). Finding a "kneedle" in a haystack: detecting knee points in system behavior. *31st International Conference on Distributed Computing Systems Workshops*.
8. Bhat, R. A. et al. Universal Dependencies Hindi HDTB treebank. github.com/UniversalDependencies/UD_Hindi-HDTB
9. Artetxe, M., Ruder, S., & Yogatama, D. (2020). On the cross-lingual transferability of monolingual representations (XQuAD). *ACL 2020*.
10. Sarvam AI (2024). Sarvam-1: the first Indian language LLM. sarvam.ai/blogs/sarvam-1
11. Speer, R. et al. `wordfreq`: word frequencies in 40+ languages.

Appendix A Running the pipeline

SCRIPT	PURPOSE
<code>01_build_corpus.py</code>	Fetch and clean the corpus. <code>--source wiki</code> streams the Hindi Wikipedia dump; <code>--source mirror</code> assembles the GitHub-mirrored corpus used here.
<code>02_tokenize.py</code>	Devanagari-aware tokenisation, frequency table, corpus statistics.
<code>03_zipf_analysis.py</code>	Baseline fits plus the nine falsification attacks.
<code>04_heaps_analysis.py</code>	Heaps fit, Kneedle flattening point, marginal yield, English control, Zipf–Heaps duality.
<code>05_cost_model.py</code>	Heaps-derived corpus tiers and the two costings.
<code>06_figures.py</code>	All twelve figures.
<code>07_report.py</code>	This document.